

Name - Preetish Khandelwal
PRN - 202501080187
EDS assignment no 2

1. Data Loading

Problem Statement: Load the dataset and display the first 5 rows to understand the structure.

```
import pandas as pd
import numpy as np

df = pd.read_csv('retail_sales_dataset.csv')
print(df.head())
```

	Transaction ID	Date	Customer ID	Gender	Age	Product	Category \
0	1	2023-11-24	CUST001	Male	34		Beauty
1	2	2023-02-27	CUST002	Female	26		Clothing
2	3	2023-01-13	CUST003	Male	50		Electronics
3	4	2023-05-21	CUST004	Male	37		Clothing
4	5	2023-05-06	CUST005	Male	30		Beauty

	Quantity	Price per Unit	Total Amount
0	3	50	150
1	2	500	1000
2	1	30	30
3	1	500	500
4	2	50	100

2. Missing Value Inspection

Problem Statement: Check for any missing values across all columns in the dataset.

```
print(df.isnull().sum())
```

Transaction ID	0
Date	0
Customer ID	0
Gender	0
Age	0
Product Category	0
Quantity	0
Price per Unit	0
Total Amount	0
dtype:	int64

3. Datetime Conversion

Problem Statement: The 'Date' column is currently an object. Convert it into a proper pandas `datetime` object.

```
df['Date'] = pd.to_datetime(df['Date'])
print(df['Date'].dtype)

... datetime64[ns]
```

4. Total Revenue Calculation

Problem Statement: Calculate the total revenue generated from all transactions recorded in the dataset.

```
total_revenue = df['Total Amount'].sum()
print(f"Total Revenue: ${total_revenue}")

... Total Revenue: $456000
```

5. Customer Demographics (Average Age)

Problem Statement: Find the average age of the customers who made purchases.

```
average_age = df['Age'].mean()
print(f"Average Customer Age: {average_age:.2f}")

... Average Customer Age: 41.39
```

6. Product Variety

Problem Statement: Determine the total number of unique product categories available in the store.

```
unique_categories = df['Product Category'].nunique()
print(f"Unique Categories: {unique_categories}")

... Unique Categories: 3
```

7. Sales Volume by Category

Problem Statement: Find the total number of items (quantity) sold for each product category.

```
quantity_per_category = df.groupby('Product Category')['Quantity'].sum()
print(quantity_per_category)

... Product Category
Beauty      771
Clothing    894
Electronics 849
Name: Quantity, dtype: int64
```

8. Gender-based Sales Analysis

Problem Statement: Calculate the total sales amount contributed by each gender.

```
sales_by_gender = df.groupby('Gender')['Total Amount'].sum()
print(sales_by_gender)
```

```
... Gender
Female    232840
Male      223160
Name: Total Amount, dtype: int64
```

9. High-Value Transactions

Problem Statement: Identify the top 5 transactions that resulted in the highest 'Total Amount'.

```
top_5_sales = df.nlargest(5, 'Total Amount')
print(top_5_sales)
```

```
...   Transaction ID   Date Customer ID Gender Age Product Category \
14         15 2023-01-16   CUST015  Female  42   Electronics
64         65 2023-12-05   CUST065   Male  51   Electronics
71         72 2023-05-23   CUST072  Female  20   Electronics
73         74 2023-11-22   CUST074  Female  18    Beauty
88         89 2023-10-01   CUST089  Female  55   Electronics

   Quantity  Price per Unit  Total Amount
14         4             500           2000
64         4             500           2000
71         4             500           2000
73         4             500           2000
88         4             500           2000
```

10. Time-Series Feature Engineering

Problem Statement: Extract the 'Month' and 'Year' from the 'Date' column and add them as new separate columns.

```
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
print(df[['Date', 'Year', 'Month']].head())
```

```
...   Date      Year  Month
0  2023-11-24  2023     11
1  2023-02-27  2023      2
2  2023-01-13  2023      1
3  2023-05-21  2023      5
4  2023-05-06  2023      5
```

11. Average Transaction Value by Category

Problem Statement: Calculate the average 'Total Amount' spent per transaction for each product category.

```

avg_spent_category = df.groupby('Product Category')['Total Amount'].mean()
print(avg_spent_category)

... Product Category
Beauty      467.475570
Clothing    443.247863
Electronics 458.786550
Name: Total Amount, dtype: float64

```

12. Price Distribution (NumPy)

Problem Statement: Use NumPy to calculate the variance and standard deviation of the 'Price per Unit'.

```

price_variance = np.var(df['Price per Unit'])
price_std = np.std(df['Price per Unit'])
print(f"Variance: {price_variance}, Std Dev: {price_std}")

... Variance: 35943.03789999965, Std Dev: 189.58649187112368

```

13. Bulk Order Filtering

Problem Statement: Filter and display the transactions where the 'Quantity' of items purchased is 3 or more.

```

bulk_orders = df[df['Quantity'] >= 3]
print(bulk_orders.head())

... Transaction ID      Date Customer ID Gender Age Product Category \
0              1 2023-11-24   CUST001   Male   34         Beauty
7              8 2023-02-22   CUST008   Male   30      Electronics
9             10 2023-10-07   CUST010  Female   52         Clothing
11            12 2023-10-30   CUST012   Male   35         Beauty
12            13 2023-08-05   CUST013   Male   22      Electronics

   Quantity  Price per Unit  Total Amount  Year  Month
0          3             50           150  2023     11
7          4             25           100  2023     2
9          4             50           200  2023    10
11         3             25            75  2023    10
12         3            500          1500  2023     8

```

14. Multi-level Aggregation

Problem Statement: Group the data by both 'Product Category' and 'Gender' to see the average 'Total Amount' spent.

```
multi_group = df.groupby(['Product Category', 'Gender'])['Total Amount'].mean()
print(multi_group)
```

```
... Product Category Gender
Beauty Female 450.783133
      Male 487.127660
Clothing Female 467.097701
        Male 419.802260
Electronics Female 451.382353
            Male 466.104651
Name: Total Amount, dtype: float64
```

15. Lowest Sale Identification

Problem Statement: Find the specific transaction record that had the minimum 'Total Amount'.

```
min_sale_record = df.loc[df['Total Amount'].idxmin()]
print(min_sale_record)
```

```
... Transaction ID 44
Date 2023-02-19 00:00:00
Customer ID CUST044
Gender Female
Age 22
Product Category Clothing
Quantity 1
Price per Unit 25
Total Amount 25
Year 2023
Month 2
Name: 43, dtype: object
```

16. Monthly Revenue Trends

Problem Statement: Calculate the total revenue generated for every month available in the dataset.

```
monthly_revenue = df.groupby(['Year', 'Month'])['Total Amount'].sum()
print(monthly_revenue)
```

```
... Year Month
2023 1 35450
      2 44060
      3 28990
      4 33870
      5 53150
      6 36715
      7 35465
      8 36960
      9 23620
      10 46580
      11 34920
      12 44690
2024 1 1530
Name: Total Amount, dtype: int64
```

17. Age Group Analysis

Problem Statement: Categorize customers into age bins (18-25, 26-35, etc.) and identify which age group spent the most in total.

```
bins = [18, 25, 35, 45, 55, 65, 100]
labels = ['18-25', '26-35', '36-45', '46-55', '56-65', '65+']
df['Age Group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)
top_age_group = df.groupby('Age Group')['Total Amount'].sum().idxmax()
print(f"Top Spending Age Group: {top_age_group}")

... Top Spending Age Group: 46-55
/tmp/ipykernel_17015/94535553.py:4: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version.
  top_age_group = df.groupby('Age Group')['Total Amount'].sum().idxmax()
```

18. Correlation Check

Problem Statement: Check if there is any statistical correlation between the 'Age' of a customer and the 'Total Amount' they spend.

```
correlation = df['Age'].corr(df['Total Amount'])
print(f"Correlation between Age and Spending: {correlation}")

... Correlation between Age and Spending: -0.0605680238830457
```

19. Targeted Category Insights

Problem Statement: Count how many transactions were made in each product category specifically by 'Female' customers.

```
female_preferences = df[df['Gender'] == 'Female']['Product Category'].value_counts()
print(female_preferences)

... Product Category
Clothing      174
Electronics   170
Beauty        166
Name: count, dtype: int64
```

20. Price Manipulation (NumPy Vectorization)

Problem Statement: Use NumPy to create a new column 'Discounted Price' which represents a 10% discount on the original 'Price per Unit'.

```
df['Discounted Price'] = np.multiply(df['Price per Unit'], 0.9)
print(df[['Price per Unit', 'Discounted Price']].head())

...   Price per Unit  Discounted Price
0              50              45.0
1             500             450.0
2              30              27.0
3             500             450.0
4              50              45.0
```